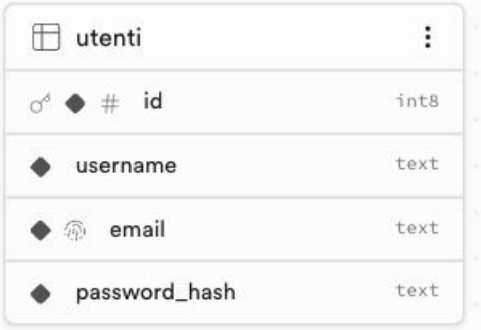
Parte 1
Autenticazione e Sessioni

Autenticazione

Per l’autenticazione dovremo ovviamente salvare in qualche modo gli utenti registrati al sito. Per fare ciò possiamo aggiungere una tabella ‘Utenti’ nel database.

Per l’esempio che vi ho preparato la tabella sarà molto semplice e conterrà solo: id utente (generato in automatico da Supabase), username, email e un hashing della password (non possiamo salvare la password in chiaro!).



Le richieste HTTP

- **Registrazione:** per registrare un nuovo utente il client dovrà mandare una richiesta http contentente i dati utente. A sua volta il server deve effettuare una insert nella tabella Utenti.
- **Login:** se l'utente è già registrato allora dovrà mandare una richiesta di login contenente email e password. A sua volta il server deve effettuare una select per verificare se quell'utente c'è e verificare se la password corrisponde.
- **Profilo:** se l'utente è già loggato (quindi ha una sessione attiva) deve poter accedere ai suoi dati utente. Per farlo invia una richiesta al client. Il client può mandare questa richiesta anche se non è loggato ma in questo caso verrà reindirizzato alla pagina di Login.
- **Log out:** se l'utente è già loggato (quindi ha una sessione attiva) deve poter effettuare il log out che termina l'attuale sessione.

Sessioni

Le sessioni sono associate a due cose:

- un cookie che viene immagazzinato nel browser del client e che contiene un codice di accesso (in questo modo il server può “ricordarsi” quello specifico client).
- una entry in una tabella nel database (questa tabelle l'ho chiamata session), questa entry contiene:
  - lo stesso codice di accesso che si trova nel cookie
  - dati dell'utente di base in formato JSON (in alternativa potreste immagazzinare solo l'id utente e poi dare un join con la tabella utenti per ottenere gli altri dati sull'utente)
  - un timestamp di scadenza (le sessioni non durano per sempre)

Le librerie che usiamo

Siccome immagazzinare le sessione nel database e gestire i cookie è complicato utilizziamo le seguenti librerie:

- **express-session:** libreria specifica per express, gestisce cookie e sessioni in modo automatico.
- **connect-pg-simple:** gestisce in modo automatico l'archiviazione delle sessioni nel database
- **pg:** stabilisce un endpoint di connessione verso il database

ATTENZIONE: connect-pg-simple e pg sono specificiche per database di tipo PostgreSQL; se usate altri database dovrete utilizzare altre librerie che fanno la stessa cosa.

La tabella session

L’uso delle librerie sovra elencate impone che la tabella session nel database abbia un formato preciso, in particolare sono presenti i seguenti attributi (gli stessi elencati nella slide precedente):

- **“sid”**: codice di sessione, lo stesso che viene immagazzinato nel cookie. Questo codice è generato in automatico da express-session
- **“sess”**: è un codice JSON contentente dati utente di base che inseriamo noi. ATTENZIONE: qui non sono presenti tutti i dati utente, quelli sono immagazzinati dentro la tabella Utenti.
- **“expire”**: generato secondo l’età del cookie che impostiamo noi.

Creazione della tabella session

Ecco il codice PostrgreSQL per creare la tabella session:

```
CREATE TABLE "session" (
 "sid" varchar NOT NULL COLLATE "default",
 "sess" json NOT NULL,
 "expire" timestamp(6) NOT NULL
)
WITH (OIDS=FALSE);
ALTER TABLE "session" ADD CONSTRAINT "session_pkey" PRIMARY KEY ("sid") NOT DEFERRABLE INITIALLY IMMEDIATE;
CREATE INDEX "IDX_session_expire" ON "session" ("expire");
```

